A Deterministic Numerical Stack

N. E. Davis ~lagrev-nocfep
ZeroAttest, Groundwire Foundation

Abstract

Contents

1 Introduction

Urbit is a deterministic functional operating system whose state is a pure function of its event log (~sorreg-namtyv et al., 2016). An event log replay must recover the state exactly, on any conforming runtime, indefinitely. This is not a performance

Manuscript submitted for review.

Address author correspondence to ~lagrev-nocfep.

nicety; it is the property that makes the system auditable, upgradeable, and peer-verifiable. Determinism is a load-bearing property all the way down.

Numerical computing is one of the most slippery places for determinism to leak through its abstractions. Standard IEEE 754 floating-point arithmetic is not associative, as an example, but the situation is much more dire. The IEEE 754 standard (, 2019) specifies the result of each individual operation under a given rounding mode, but a real numerical program is a composition of operations, and the standard says little about composition. A fused multiply-add computes $a \cdot b + c$ with a single rounding where two separate operations would round twice; a vectorized dot product sums partial results in an order chosen by the hardware; and a threaded BLAS reassociates a reduction differently depending on how many cores happened to be available.[1] Each of these is standards-conformant while none is deterministic across hosts.

The treatment of floating-point representation in Hoon by `~lagrev-nocfep` (2024) established the foundation for Urbit's deterministic numerical computing stack. This article expounds both the state of the art of Urbit's basic numerical packaging and the next layer: dense linear algebra, where the nondeterminism of conventional BLAS and IEEE 754 floating-point mathematics are acute and consequential. We introduce SoftBLAS,[2] a software-defined BLAS package; SoftUnum,[3] a software-defined universal number (unum) package; and other components of a C-based numerical stack that is deterministic and compliant with the Hoon specification. We further describe the "ray" data structure which facilitates the type dispatch for different numeric representations while preserving identity between Nock-specified atoms and C-recognized arrays.

The central conceit is architectural. In conventional high-performance computing, C (or Fortran) takes primacy: the C implementation is what runs, and correctness is in some sense

[1] Urbit computing is single-threaded which removes one possible source of nondeterminism.

[2] `urbit/SoftBLAS`

[3] `sigilante/SoftUnum`

whatever the C does.[4] Urbit inverts this relationship. The Hoon/Nock ISA implementation is the specification; the C jet is an optimization that is *obligated* to produce the specification's exact output for every input, and should be discarded silently whenever it cannot. Determinism then is not merely a testable, desirable property, but a mandate the Nock-side architecture cannot express the violation of.

2 Background and Literature

Although the solution of partial differential equations is one of the oldest significant applications of computing,[5] no standard library emerged for decades—unsurprising given the churn of hardware architectures, the diversity of numerical methods, the lack of a common interface definition, the rapid evolution of programming languages, and the active development of linear algebraic algorithms.[6] Given that situation, rigorous bitwise determinism was a concern of vanishingly small importance, whether cross-platform or even for serial runs on the same machine. However, over time the advent of high-performance computing, the emergence of BLAS as a pseudo-standard, and the increasing importance of reproducibility in numerical computing have made determinism both more pressing and more achievable.

2.1 blas as a pseudo-standard

The Basic Linear Algebra Subprograms originate with Lawson, Hanson, Kincaid, and Krogh's 1979 specification of vector operations on strided arrays (Lawson et al., 1979), later extended to matrix-vector (Level 2) and matrix-matrix (Level 3) routines. The levels track both chronology and (naïve) asymptotic cost: Level 1 is $O(n)$, Level 2 is $O(n^2)$, Level 3 is $O(n^3)$. BLAS is less a standard than a pseudo-standard—a widely honored set of names and calling conventions (`?axpy`, `?gemv`, `?gemm`, with

[4] Verification and validation apply, of course.

[5] **TODO** (**TODO**).

[6] Even the size of a byte took decades to standardize.

type prefixes `s`, `d`, `c`, `z`) with many independent implementations. (The original BLAS predated floating-point standardization as IEEE 754 by several years.) That plurality is exactly the problem for a consensus system: two conforming BLAS libraries routinely disagree in the low bits, and the same library disagrees with itself across machines and thread counts. Despite this fuzziness, BLAS quickly took its place as the bedrock layer of numerical and scientific computation.

2.2 Sources of floating-point nondeterminism

Cross-host divergence in a BLAS computation has a small number of recurring causes:

- *Reduction order.* Floating-point addition is not associative, so a dot product or a `gemm` inner loop summed in a different order yields a different rounded result; tiled and multithreaded implementations choose that order at run time.

- *Fused multiply-add.* Hardware that contracts $a \cdot b + c$ to one rounding produces different bits than two roundings, and whether contraction happens is a compiler- and target-dependent decision.

- *Vectorization and instruction selection.* SIMD horizontal sums, `x87` 80-bit intermediates, and differing `libm`-style kernels for fast paths all perturb the last bits.

- *Transcendental functions.* The elementary functions are not specified to the bit by IEEE 754 at all, and every platform `libm` differs in its implementation.

`~lagrev-nocfep` (2024) further enumerated other reasons that contemporary floating-point mathematics cannot be considered deterministic even when single-threaded on a particular hardware target and instruction set architecture.

2.3 Reproducibility

Numerical determinism, or for our purposes synonymously reproducibility, requires that a computation yield the same result to the bit regardless of the platform on which it is carried out. This would not necessarily require the same algorithmic implementation in all cases (since all correct algorithms should converge on the true value given enough runway), but in practice floating-point quirks force this consideration in almost all cases.

Projects like ReproBLAS have yielded reproducible summation (independent of summation order) through error-free transformations and pre-rounding (Demmel and Nguyen, 2013). Intel's Conditional Numerical Reproducibility offers run-to-run consistency within a fixed code path under documented constraints (Intel Corporation, 2023). The PLASMA and MAGMA projects (Agullo et al., 2009) took the opposite design stance for performance: dynamic, dependency-driven scheduling that deliberately does not fix reduction order, which is excellent for throughput and fatal for bit-reproducibility. On the scalar side, correctly rounded elementary-function libraries—CR-LIBM (Daramy et al., 2003), RLIBM (Lim and Nagarakatte, 2021), and the CORE-MATH project (The CORE-MATH Project, 2022)—show that the transcendentals can be pinned down to the last bit, at a cost.

Urbit is distinguished by its requirement that determinism be not one option among many but a first-class feature of the OS and runtime. This requirement is structurally enforced by the Nock ISA–jetted runtime relationship. Jets utilize relatively faster code paths and reductions but produce the identical result as the slow path: the specification is authoritative. Thus linear algebra techniques, even those following the BLAS pseudo-standard, must follow a reproducible fast path on all supported architectures.

3 Deterministic Architecture

A Nock computation is a pure function from noun to noun. A *jet* is a native routine registered against a named Hoon core; when the interpreter evaluates that core, it may run the jet instead of the Nock. The iron discipline that makes this sound is simple: the jet must produce the same noun as the Hoon, for every input, or it is a bug in the runtime. A divergence between jet and specification is termed a *jet mismatch* and is treated as a correctness failure, not a numerical tolerance.

The Hoon (or its compiled result as Nock) is therefore the specification, in the strong sense that it defines what the right answer *is*. The C jet is an accelerant with no semantic authority. In some cases, the jet may decline: if a native routine returns the sentinel `u3_none`, the interpreter falls through and evaluates the Hoon directly. The consequence for numerics is that an arithmetic kind with no jet, or with a jet that punts on some inputs, is not unsupported—it is merely slow. The specification still runs, and the answer is still defined and still deterministic, because Hoon over exact nouns is deterministic by construction.

This crystallizes, or perhaps even inverts, the usual relationship between correctness and performance. In conventional HPC, one has an optimized C or Fortran program, the optimized C arguably *is* the program and its source reference implementation is documentation (to say nothing of its documentation). There may of course be bugs in the compiled code, but the optimized program is the concrete artifact. By contrast in Urbit, the optimized C program must justify itself against an independent authoritative definition. Determinism thus follows from two facts: the specification is a pure function of exact data, and the jet is contractually bit-identical to the specification. A host without the jet computes the same bits more slowly; a host with a divergent jet is non-conforming and must be repaired.

For our linked implementations like SoftBLAS, this means the library is never permitted to be "approximately" the specification. A `gemm` jet that summed its inner products in a hardware-

chosen order would be a mismatch even if every individual rounding were IEEE 754-correct, because the Hoon specification sums them in one fixed order and the jet must reproduce *that* number.[7] Our BLAS must satisfy the constraint of determinism to be admissible as a jet at all; thus, SoftBLAS was developed on top of SoftFloat (Hauser, 2018). This yields correctness and consistency rather than hardware-optimized performance.

3.1 The numerical stack

Urbit natively supports IEEE 754 floating-point mathematics in its `%base` distribution, and through the supplied numerical stack additionally supports unums (posits and quires); complex IEEE 754; two's complement integers; and fixed-point mathematics.

The numerical libraries are layered as follows; each layer is Hoon-normative with optional, independently verifiable C jets.

1. *Libraries.* Imported from `/lib` at compile time.
   (a) `/lib/math`. The basic IEEE 754 arithmetic and transcendental functions library supports four precisions (16-bit `@rh`, 32-bit `@rs`, 64-bit `@rd`, and 128-bit `@rq`) across a battery of functions like exponentiation and logarithm, trigonometric and inverse-trigonometric functions, and rounding and comparison utilities. Transcendental functions are implemented as Chebyshev minimax polynomials with Cody–Waite range reduction (Cody and Waite, 1980). Jets run on SoftFloat (Hauser, 2018). Originally composed by `~lagrev-nocfep` and `~mopfel-winrux`.
   (b) `/lib/twoc`. A two's-complement integer arithmetic package supporting scalar and array operations TODO. Jetted with an internal library. Originally composed by `~litlep-nibbyt`.

[7] Summation will necessarily exhibit order-dependent drift at the bit level.

   (c) `/lib/fixed`. Fixed-point (Q-format) arithmetic at power-of-two total widths. Unjetted beyond standard integer arithmetic jets because de facto integer mathematics hold. Originally composed by `~lagrev-nocfep`.

   (d) `/lib/unum`. 2022-standard posit arithmetic (, 2022) at five widths (8-bit `@rpb`; 16-bit `@rph`; 32-bit `@rps`; 64-bit `@rpd`; 128-bit `@rpq`, i.e. posit8 through posit128), $es = 2$ throughout, with quire accumulation. See Section **??** on oracle coverage. Jetted with SoftUnum. Originally composed by `~lagrev-nocfep`.

   (e) `/lib/complex`. Complex arithmetic over the four IEEE 754 precisions, riding on the scalar float layer. Originally composed by `~lagrev-nocfep`.

2. *Lagoon (Linear AlGebra in hOON).* A rank-N array library (a `ray`) carrying a `kind` field that dispatches to the appropriate arithmetic: `%i754`, `%uint`, `%int2`, `%unum`, `%cplx`, `%fixp`. Storage is row-major and identical in memory layout to underlying runtime arrays; the `%i754` Level-3 path is jetted through SoftBLAS and the `%unum` path is jetted through SoftUnum.

3. *Saloon (Scientific ALgorithms in hOON).* Scientific algorithms over Lagoon rays. Types correspond to the Lagoon field. Element-wise transcendentals are dispatched through `/lib/math`, and dense routines such as the Jacobi symmetric eigendecomposition.

4. *Maroon (MAchine leaRning in hOON).* An inchoate machine learning library built atop Lagoon and Saloon.

The single most consequential design choice is Lagoon's `kind` dispatch with fall-through (Listing 3.1). One basic array type serves six arithmetics; adding a seventh requires only a Hoon implementation, and a jet for it is optional and separately auditable. This is architecturally distinct from NumPy (which supplies one `dtype` per array with no in-band kind tag); from Julia (which has compile-time specialization per concrete

type); and from the APL/J/K tradition (which supply a uniform numeric tower with no explicit kind dispatch). Because an un-jetted kind still runs its Hoon, the specification is authoritative for every kind, jetted or not. Two other features of `$ray`'s design call attention: the MSB pinned `1` bit, which preserves otherwise-stripped leading zeroes in atoms and thus makes the atom representation exactly match the expected C memory layout of an array; and the `tail=*` arbitrary noun specialization field, which preserves jet axis addresses in case of future extension.

Listing 1: Lagoon base types. From `/sur/lagoon`.

```
+$  ray                 ::  $ray:  n-dimensional array
  $:  =meta             ::  descriptor
      data=@ux          ::  data, row-major order
  ==
::
+$  prec  [a=@ b=@]     ::  fixed-point precision
+$  meta                ::  $meta:  metadata for a $ray
  $:  shape=(list @)    ::  list of dimension lengths
      =bloq             ::  logarithm of bitwidth
      =kind             ::  name of data type
      tail=*            ::  per-kind specialization data
  ==
::
+$  kind                ::  $kind:  type of array
    scalars
  $?  %i754             ::  IEEE 754 float
      %uint             ::  unsigned integer
      %int2             ::  2s-complement integer
    (/lib/twoc)
      %unum             ::  unum/posit (/lib/unum)
      %cplx             ::  complex (/lib/complex)
      %fixp             ::  fixed-point Q a.b
    (/lib/fixed)
  ==
::
+$  baum                ::  $baum:  ndray with metadata
  $:  =meta             ::
      data=ndray        ::
  ==
```

```
::
+$  ndray               ::  $ndray:  n-dim array as list
    $@  @               ::  single item
    (list ndray)        ::  nonempty list of children
```

8

4 Conclusion

To summarize, why did you write it? Why do we care? What impact should it have on Urbit development?

Your bibliography is a separate BibTeX file. We use the `plainnat` bibliography style. You can use `natbib` citation commands like `\citep{wikipedia}` for parenthesized references. Use `\citet{wikipedia}` for inline references. You can also use `\citeauthor{wikipedia}` for the principal author's name.

"You can use traditional TeX—or LaTeX—representations" (Varney, 1987).

"Or you can use fancy quotes—and symbols."

End the article with the tombstone.⊠

References

Agullo, Emmanuel et al. (2009). "Numerical Linear Algebra on Emerging Architectures: The PLASMA and MAGMA Projects." In: *Journal of Physics: Conference Series* 180, p. 012037. DOI: 10.1088/1742-6596/180/1/012037.

Cody, William J. and William M. Waite (1980). *Software Manual for the Elementary Functions*. Prentice-Hall.

Daramy, Catherine et al. (2003). "CR-LIBM: A Correctly Rounded Elementary Function Library." In: *Proc. SPIE 5205, Advanced Signal Processing Algorithms, Architectures, and Implementations* xiii. DOI: 10.1117/12.505591.

[8] For ':: of the packed-pair element width (both components :: combined) — e.g., '@cs' (two '@rs' components) has :: bloq=6, while each component '@rs' has bloq=5.

Demmel, James and Hong Diep Nguyen (2013). "Fast Reproducible Floating-Point Summation." In: *2013 IEEE 21st Symposium on Computer Arithmetic (ARITH)*. ReproBLAS project, pp. 163–172. DOI: 10.1109/ARITH.2013.9.

Hauser, John R. (2018). *Berkeley SoftFloat, Release 3e*. http://www.jhauser.us/arithmetic/SoftFloat.html.

IEEE Standard for Floating-Point Arithmetic (2019). IEEE Std 754-2019. IEEE. DOI: 10.1109/IEEESTD.2019.8766229.

Intel Corporation (2023). *Conditional Numerical Reproducibility (CNR) in Intel oneAPI Math Kernel Library*. Intel Developer Documentation.

~lagrev-nocfep, N. E. Davis (2024). "The Desert of the Reals: Floating-Point Arithmetic on Deterministic Systems." In: *Urbit Systems Technical Journal* 1.1, pp. 93–131.

Lawson, C. L. et al. (1979). "Basic Linear Algebra Subprograms for Fortran Usage." In: *ACM Transactions on Mathematical Software* 5.3, pp. 308–323. DOI: 10.1145/355841.355847.

Lim, Jay P. and Santosh Nagarakatte (2021). "High Performance Correctly Rounded Math Libraries for 32-bit Floating Point Representations." In: *Proc. 42nd ACM SIGPLAN Conf. on Programming Language Design and Implementation (*pldi*)*, pp. 359–374. DOI: 10.1145/3453483.3454049.

~sorreg-namtyv, Curtis Yarvin et al. (2016). *Urbit: A Solid-State Interpreter*. Whitepaper. Tlon Corporation. URL: https://media.urbit.org/whitepaper.pdf (visited on ~2024.1.25).

Standard for Posit Arithmetic (2022) (2022). es = 2 fixed across all widths. Posit Working Group. URL: https://posithub.org/docs/posit_standard-2.pdf.

The CORE-MATH Project (2022). *CORE-MATH: Correctly Rounded Mathematical Functions*. https://core-math.gitlabpages.inria.fr/.

Varney, Jim (1987). *Ernest Goes to Camp*. Film. Directed by John R. Cherry III. Produced by Stacy Williams. Written by John R. Cherry III and Coke Sams. Starring Jim Varney, Victoria Racimo, John Vernon, Iron Eyes Cody, Lyle Alzado, Gailard Sartain, Daniel Butler, Patrick Day, Scott

Menville, Jacob Vargas, and Todd Loyd. Buena Vista Pictures. URL: https://www.imdb.com/title/tt0092974/ (visited on ~2019.4.1).